Machine Learning II

Aula 2 — Fundamentos de Deep Learning

Daniel Yukimura, Francisco Ganacim
IMPA

18 de março de 2026

Sobre o curso

- Página do curso: https://ganacim.github.io/ml2-2026/
- Monitoria IMPA Tech
  - Ricardo Arruda
  - Marcelo Carneiro
- Monitoria IMPA
  - Melvin Poveda

Relembrando

Feedforward Neural Network (FNN)

$$\mathbf{x} \xrightarrow{\mathbf{W}^{(1)}} \mathbf{h}^{(1)} \xrightarrow{\mathbf{W}^{(2)}} \mathbf{h}^{(2)} \cdots \rightarrow \mathbf{h}^{(L-1)} \xrightarrow{\mathbf{W}^{(L)}} f_\theta$$

$$f_\theta(\mathbf{x}) = \mathbf{W}^{(L)} \sigma\left(\mathbf{W}^{(L-1)} \sigma\left(\ldots \sigma\left(\mathbf{W}^{(1)}\mathbf{x}\right)\right)\right)$$

não-linearidade

linearidade

Relembrando

Aprendizado supervisionado

Dado um conjunto de dados $(x_i, y_i)_{i=1}^n$, queremos os pesos $\theta = \{W_1, W_2, \ldots, W_L\}$ da rede que **atingem o menor valor de risco**

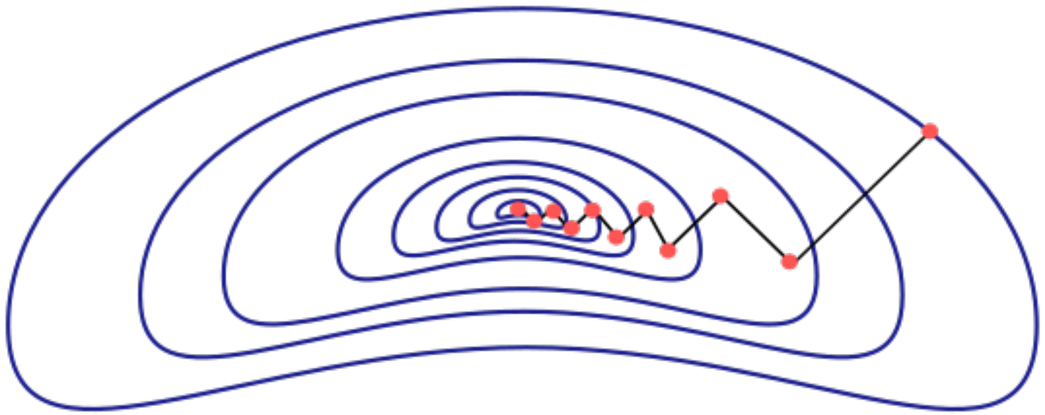
$$\begin{aligned}\hat{\theta} &= \underset{\theta \in \Theta}{\textbf{argmin}}\, R_n(\theta) \\ &= \underset{\theta \in \Theta}{\textbf{argmin}} \frac{1}{n} \sum_{i=1}^{n} \ell\left(f_\theta(x_i), y_i\right)\end{aligned}$$

Relembrando

O algoritmo de **gradiente descendente** clássico consiste na iteração

$$\theta_{t+1} = \theta_t - \alpha \nabla L(\theta_t)$$

para alguma configuração inicial θ_0 e uma taxa de aprendizado $\alpha > 0$.



Relembrando

Stochastic Gradient Descent (SGD):

- Em cada iteração $t > 0$ sorteamos um conjunto $S \subset [n]$ de índices ($|\mathscr{D}| = n$) e calculamos o gradiente de um *minibatch*

$$\hat{g}_S(\theta) = \nabla \hat{L}_S(\theta) \qquad \hat{L}_S(\theta) = \frac{1}{|S|} \sum_{i \in S} \ell(f(x_i, \theta), y_i)$$

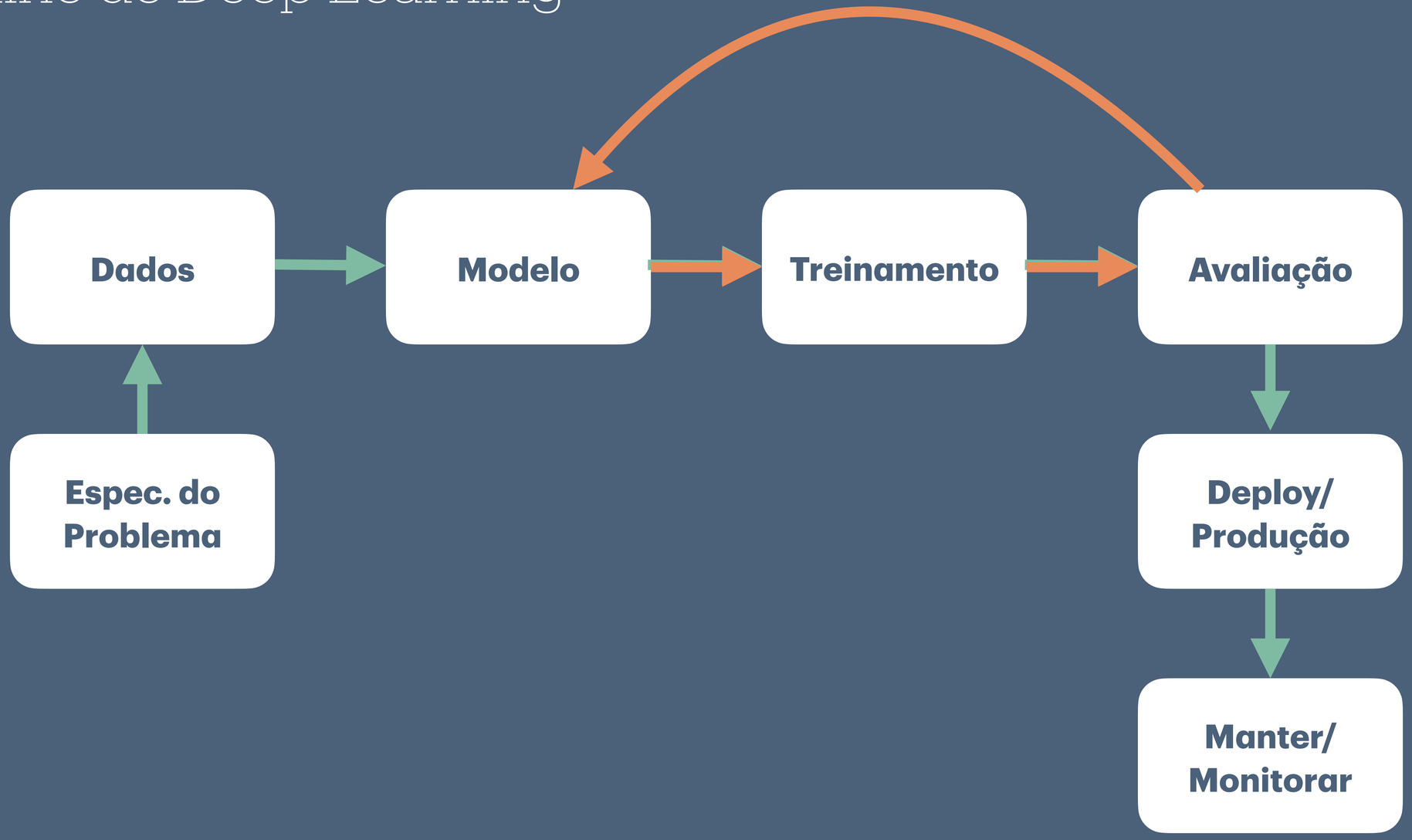
- A iteração é dada por

$$\theta_{t+1} = \theta_t - \alpha\, \hat{g}_S(\theta_t)$$

Aula 2: Fundamentos de Deep Learning

Como fazer isso na prática?

O pipeline de Deep Learning



PyTorch

Pytorch é uma biblioteca do Python para construir e treinar modelos de deep learning.

- Diferenciação automática (autograd)
- Execução vetorizada eficiente (CPU/GPU)
- Modularidade
- Interface simples e flexível, próxima do Python

Como trabalhar com Dados?

Tensores e Vetorização

Como vemos dados computacionalmente?

$$(x_i, y_i)_{i=1}^{n} \sim \mathscr{D}$$

- $x_i \in \mathbb{R}^d$ — lista de números
- $y_i =$ saída/rótulo

logo-centro-pi.png

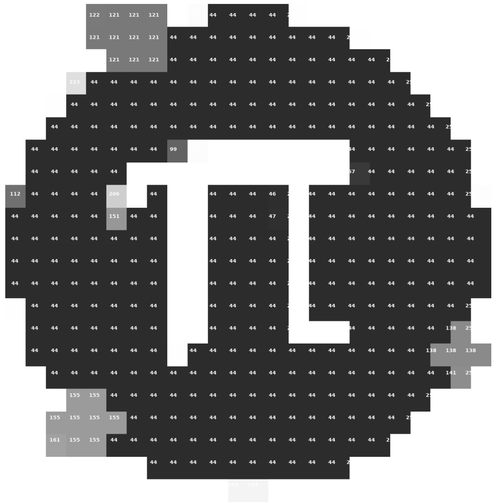
Como trabalhar com Dados?

Tensores e Vetorização

Como vemos dados computacionalmente?

$$(x_i, y_i)_{i=1}^{n} \sim \mathscr{D}$$

- $x_i \in \mathbb{R}^d$ — lista de **números**
- $y_i =$ saída/rótulo
- armazenados como `float32`



logo-centro-pi.png

Como trabalhar com Dados?

Tensores e Vetorização

Como vemos dados computacionalmente?

$$(x_i, y_i)_{i=1}^{n} \sim \mathscr{D}$$

- $x_i \in \mathbb{R}^d$ — lista de **números**
- $y_i =$ saída/rótulo
- armazenados como `float32`

255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255	255
255	255	255	255	122	121	121	121	255	252	44	44	44	44	254	255	255	255	255	255	255	255	255	255
255	255	255	255	121	121	121	121	44	44	44	44	44	44	44	44	44	254	255	255	255	255	255	255
255	255	255	255	255	121	121	121	44	44	44	44	44	44	44	44	44	44	44	255	255	255	255	255
255	255	255	223	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	255	255	255	255
255	255	254	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	255	255	255
255	255	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	255	255
255	44	44	44	44	44	44	44	99	252	255	255	255	255	255	255	255	44	44	44	44	44	44	255
255	44	44	44	44	44	255	255	255	255	255	255	255	255	255	255	255	57	44	44	44	44	44	255
112	44	44	44	44	206	255	44	255	255	44	44	44	46	255	44	44	44	44	44	44	44	44	253
44	44	44	44	44	151	44	44	255	255	44	44	44	47	255	44	44	44	44	44	44	44	44	44
44	44	44	44	44	44	44	44	255	255	44	44	44	44	255	44	44	44	44	44	44	44	44	44
44	44	44	44	44	44	44	44	255	255	44	44	44	44	255	44	44	44	44	44	44	44	44	44
44	44	44	44	44	44	44	44	255	255	44	44	44	44	255	44	44	44	44	44	44	44	44	44
254	44	44	44	44	44	44	44	255	255	44	44	44	44	255	44	44	44	44	44	44	44	44	255
255	44	44	44	44	44	44	44	255	255	44	44	44	44	255	255	255	44	44	44	44	44	138	255
255	44	44	44	44	44	44	44	254	44	44	44	44	44	44	44	44	44	44	44	44	138	138	138
255	255	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	141	255
255	255	255	155	155	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	44	255	255	255
255	255	155	155	155	155	44	44	44	44	44	44	44	44	44	44	44	44	44	44	255	255	255	255
255	255	161	155	155	44	44	44	44	44	44	44	44	44	44	44	44	44	44	255	255	255	255	255
255	255	255	255	255	255	255	44	44	44	44	44	44	44	44	44	44	255	255	255	255	255	255	255
255	255	255	255	255	255	255	255	255	255	255	244	244	255	255	255	255	255	255	255	255	255	255	255

logo-centro-pi.png

Tensores

Arrays multidimensionais

- Tensores são uma generalização de vetores e matrizes para dimensões maiores.
- Na máquina são arrays d-dimensionais.
- **Exemplo:** Uma imagem com canais RGB é um tensor 3D.

Tensores

```
import torch


x = torch.tensor([1.0, 2.0, 3.0])  # (3,)
X = torch.randn(4, 3)  # (4, 3)
```

vetor

matriz

```
from PIL import Image


img = Image.open("logo-centro-pi.png")
x = torchvision.transforms.ToTensor(img)
```

$x \in \mathbb{R}^{CxHxW}$

Tensores e Vetorização

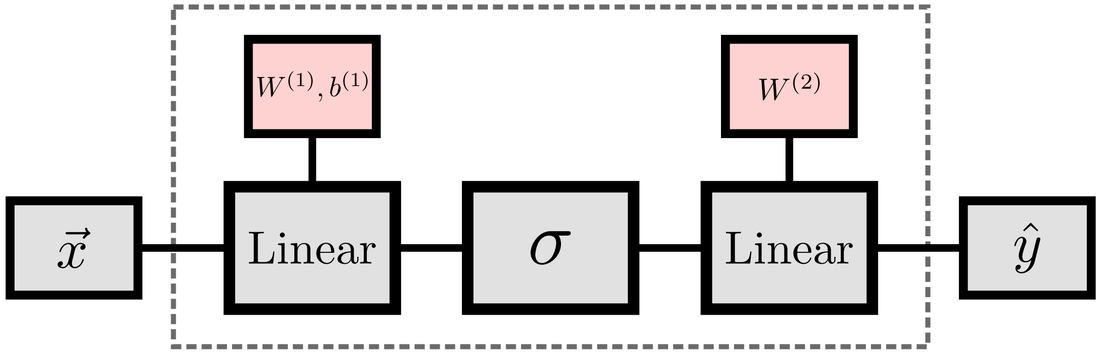
Para fazer operações em “batches” de dados de maneira eficiente (e.g. comp. distribuída) nós empilhamos os dados em tensores

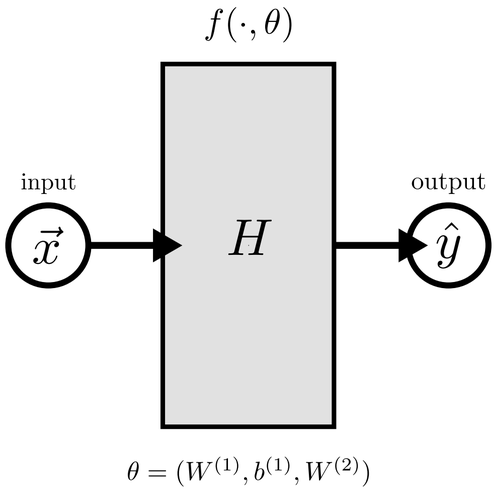
$$\{X_1, X_2, \ldots, X_k\} \longrightarrow \boxed{f_\theta} \longrightarrow \{f(X_1, \theta), f(X_2, \theta), \ldots, f(X_k, \theta)\}$$

Ex: Um batch de imagens é um tensor de dimensão 4 de tamanho $BxCxHxW$, onde B é o tamanho do batch, C = 3 se a imagem é RGB e HxW é a resolução das imagens.

Modularidade

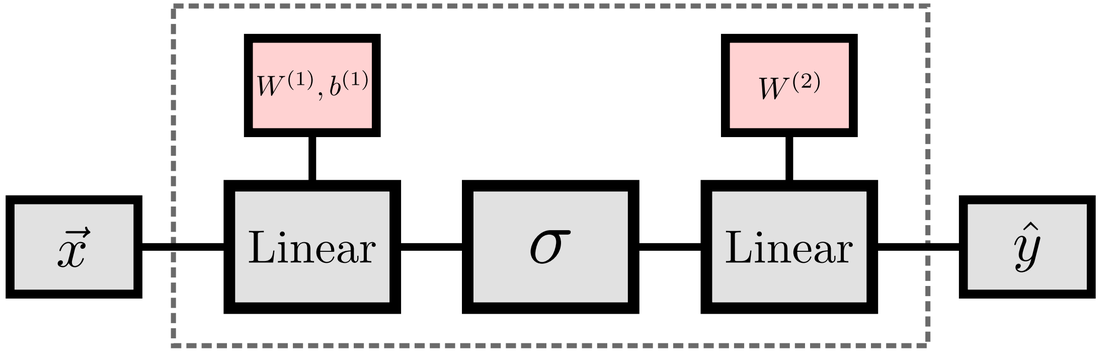
Modularidade nos permite construir estruturas complexas a partir de componentes mais simples





Modularidade

Modularidade nos permite construir estruturas complexas a partir de componentes mais simples




```
model = nn.Sequential(
    nn.Linear(10, 64),
    nn.ReLU(),
    nn.Linear(64, 1)
)
```

Modularidade

Podemos ver uma rede neural profunda como uma composição de funções

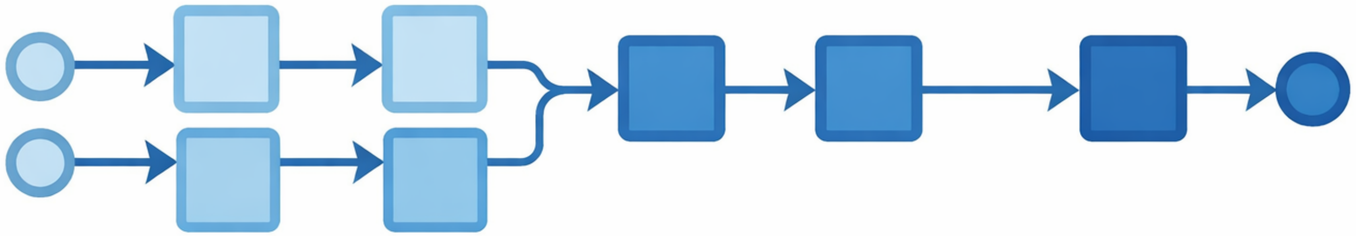
$$\mathbf{x} \longrightarrow \mathbf{h}^{(1)} \longrightarrow \mathbf{h}^{(2)} \cdots\cdots \blacktriangleright \mathbf{h}^{(L-1)} \longrightarrow f_\theta$$

$$\mathbf{h}^{(1)} = f_1(\mathbf{x}),\ \mathbf{h}^{(2)} = f_2\left(\mathbf{h}^{(1)}\right),\ \ldots,\ f_\theta = f_L\left(\mathbf{h}^{(L)}\right)$$

- Cada função transforma um tensor em outro tensor
- Em Deep Learning cada transformação é diferenciável

Grafo computacional

Trabalhamos com DAGs (grafo acíclico direcionado), onde os vértices representam transformações, e as arestas as dependências



Como treinar redes neurais?

- Gostariamos de encontrar θ tal que $f_\theta(x) \approx y$.
- Precisamos definir uma **função de perda** $L\left(f_\theta(x), y\right) \in \mathbb{R}$

```
y_hat = model(x)

loss = loss_fun(y_hat, y)
```

Como treinar redes neurais?

$$L(\hat{y}, y) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

```
loss = nn.MSELoss()(y_hat, y)
```

$$L(\hat{y}, y) = - \sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

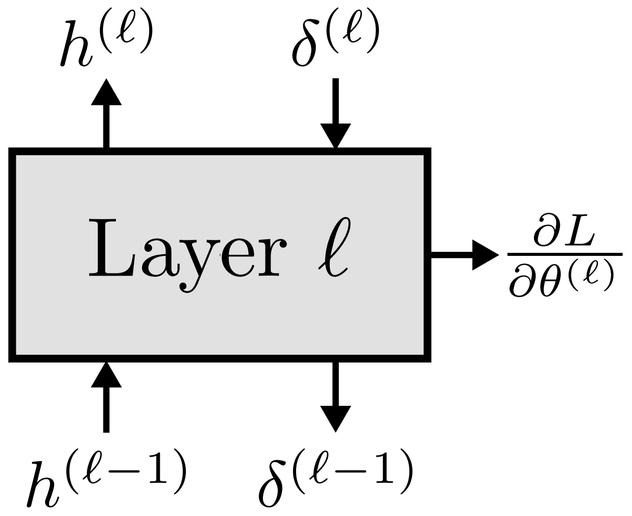
```
loss = nn.CrossEntropyLoss()(logits, y)
```

$$L(\hat{y}, y) = - \left[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}) \right]$$

```
loss = nn.BCELoss()(y_hat, y)
```

Backpropagation

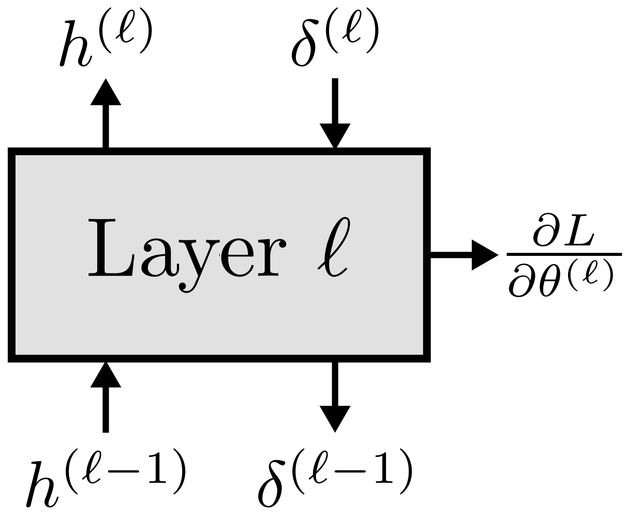
- $h^{(\ell)} = f_\ell\left(h^{(\ell-1)}, \theta^{(\ell)}\right)$
- $\dfrac{\partial L}{\partial \theta^{(\ell)}} = \dfrac{\partial L}{\partial h^{(\ell)}} \cdot \dfrac{\partial h^{(\ell)}}{\partial \theta^{(\ell)}} = \delta^{(\ell)} \dfrac{\partial h^{(\ell)}}{\partial \theta^{(\ell)}}$
- $\delta^{(\ell-1)} = \delta^{(\ell)} \cdot \dfrac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}}$



Backpropagation

Camada linear:

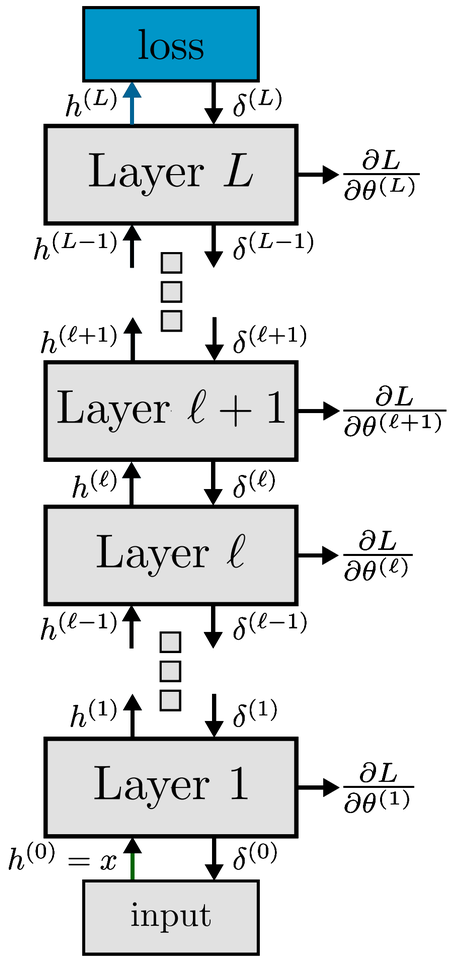
- $\mathbf{h}^{(\ell)} = \sigma\left(\mathbf{z}^{(\ell)}\right), \quad \mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}$
- $\dfrac{\partial L}{\partial \mathbf{W}^{(\ell)}} = \delta^{(\ell)} \dfrac{\partial \mathbf{h}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}}, \qquad \dfrac{\partial h_i^{(\ell)}}{\partial W_{ij}^{(\ell)}} = \sigma'\left(z_i^{(\ell)}\right) h_j^{(\ell-1)}$
- $\boldsymbol{\delta}^{(\ell-1)} = \dfrac{\partial \mathbf{h}^{(\ell)}}{\partial \mathbf{h}^{(\ell-1)}} \boldsymbol{\delta}^{(\ell)} = \left(\mathbf{W}^{(\ell)\top}\boldsymbol{\delta}^{(\ell)}\right) \odot \sigma'\left(\mathbf{z}^{(\ell-1)}\right)$



Backpropagation

- A FNN pode ser definida por um grafo computacional.
- Cada componente é diferenciável
- **Feedforward** e Backpropagation são os fluxos no grafo em ambas as direções.

$$h^{(\ell)} = f_\ell\left(h^{(\ell-1)}, \theta^{(\ell)}\right)$$

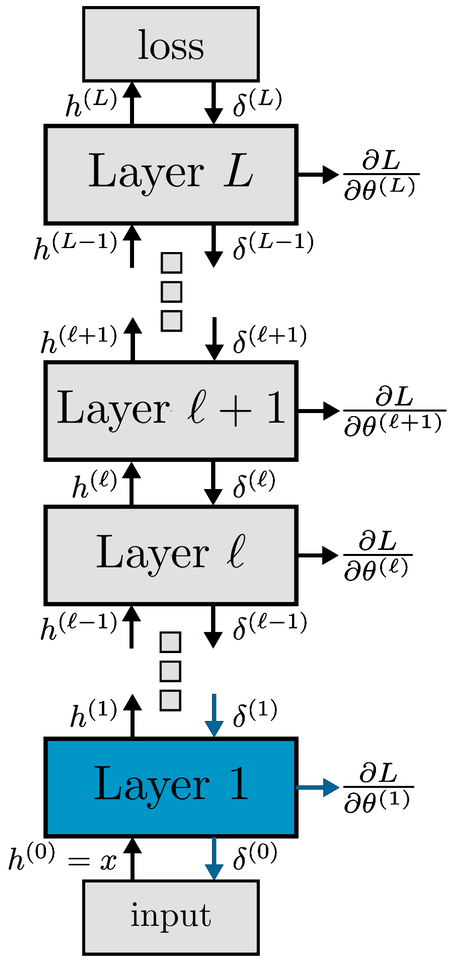


Backpropagation

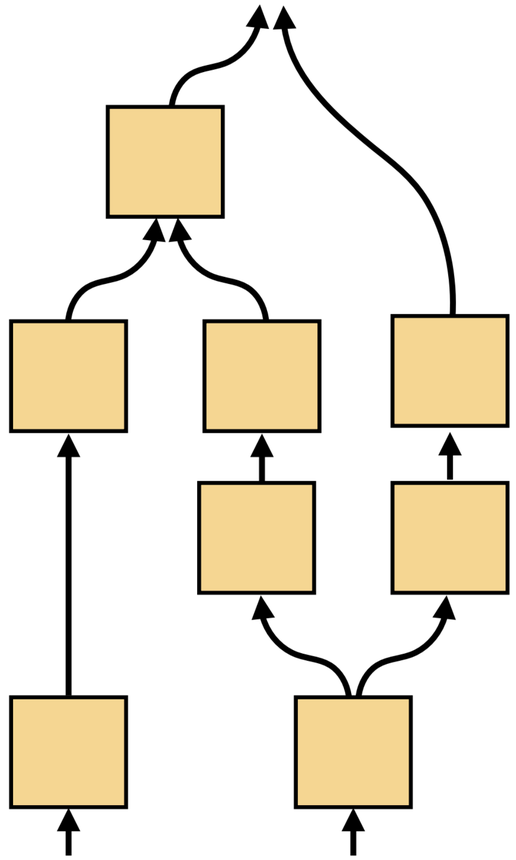
- A FNN pode ser definida por um grafo computacional.
- Cada componente é diferenciável
- Feedforward e **Backpropagation** são os fluxos no grafo em ambas as direções.

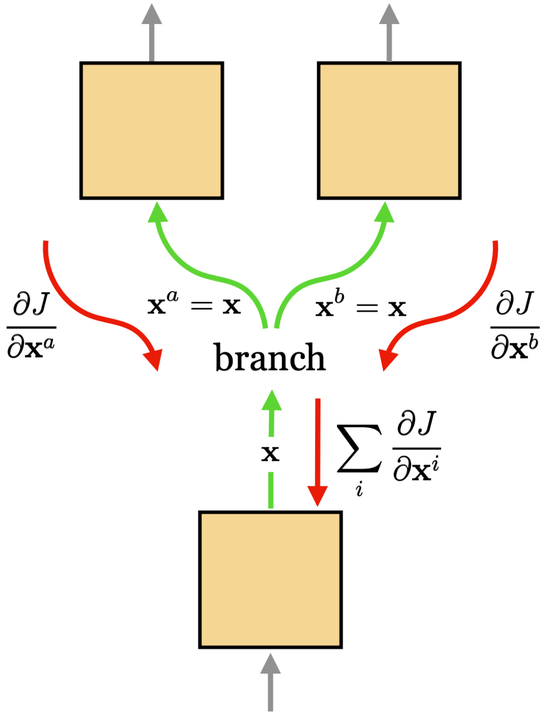
$$\delta^{(\ell-1)} = \delta^{(\ell)} \cdot \frac{\partial h^{(\ell)}}{\partial h^{(\ell-1)}}$$

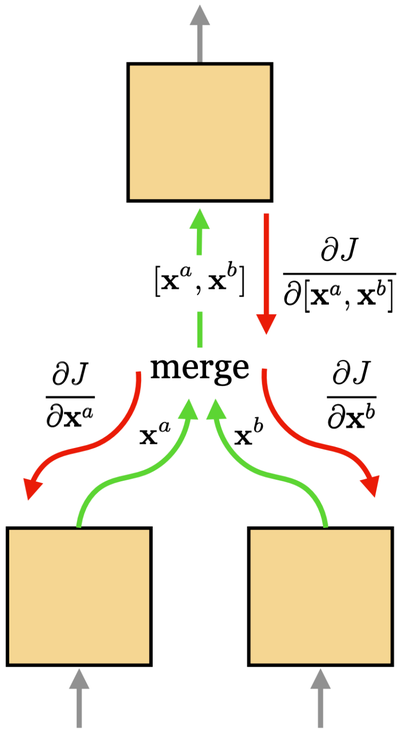
$$\frac{\partial L}{\partial \theta^{(\ell)}} = \delta^{(\ell)} \frac{\partial h^{(\ell)}}{\partial \theta^{(\ell)}}$$



Autograd




$\frac{\partial J}{\partial \mathbf{x}^a}$
$\mathbf{x}^a = \mathbf{x}$
$\mathbf{x}^b = \mathbf{x}$
$\frac{\partial J}{\partial \mathbf{x}^b}$
branch
$\mathbf{x}$
$\sum_i \frac{\partial J}{\partial \mathbf{x}^i}$


$[\mathbf{x}^a, \mathbf{x}^b]$
$\frac{\partial J}{\partial [\mathbf{x}^a, \mathbf{x}^b]}$
$\frac{\partial J}{\partial \mathbf{x}^a}$
merge
$\frac{\partial J}{\partial \mathbf{x}^b}$
$\mathbf{x}^a$
$\mathbf{x}^b$

Loop de Treinamento

$$\theta \leftarrow \theta - \alpha \nabla_\theta L\left(f_\theta(x), y\right)$$

```
for epoch in range(epochs):
    y_hat = model(x)
    loss = criterion(y_hat, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Forward

Perda

Backward

Update

Loop de Treinamento

$$\theta \leftarrow \theta - \alpha \nabla_\theta L\left(f_\theta(x), y\right)$$

```
for epoch in range(epochs):
    y_hat = model(x)
    loss = criterion(y_hat, y)


    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Loop de Treinamento

$$\theta \leftarrow \theta - \alpha \nabla_\theta L\left(f_\theta(x), y\right)$$

```
for epoch in range(epochs):
    for x_batch, y_batch in loader:
        y_hat = model(x_batch)
        loss = criterion(y_hat, y_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Loop de Treinamento

```
dataset = MyDataset(X, y)

loader = DataLoader(dataset, batch_size=32, shuffle=True)


for epoch in range(epochs):

    for x_batch, y_batch in loader:

        y_hat = model(x_batch)

        loss = criterion(y_hat, y_batch)

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()
```

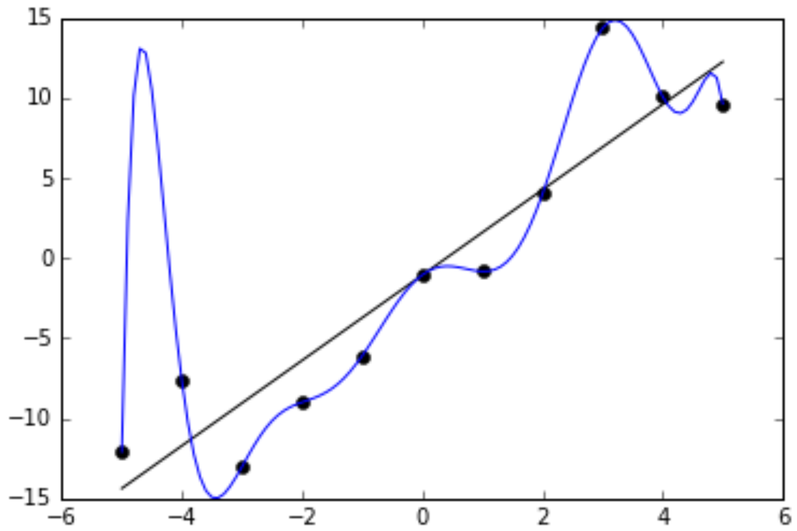
Datasets e DataLoaders

```
from torch.utils.data import Dataset, DataLoader

class MyDataset(Dataset):
    def __init__(self, X, y):
        self.X = X
        self.y = y

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

Avaliação

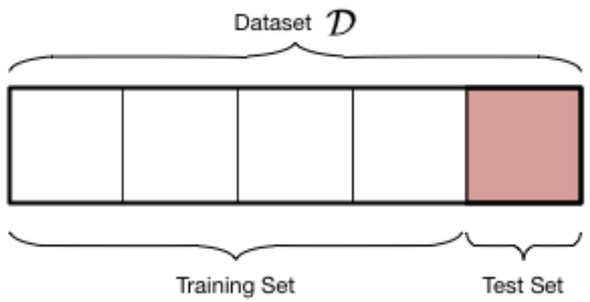
Treinar bem ≠ Generalizar bem

- O Modelo pode memorizar
- Erro de treino cair não implica desempenho real



Avaliação

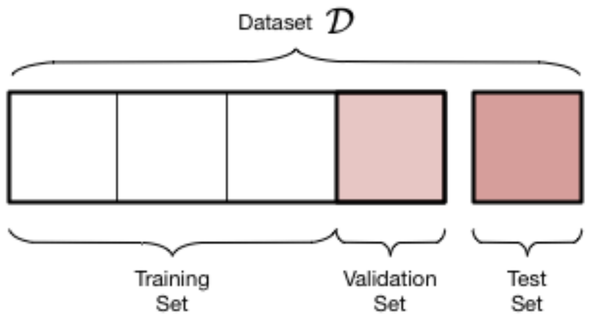
- Para avaliar generalização, o modelo precisa ser avaliado em um conjunto de dados não visto durante o treino



- Porém, a escolha de **hyperparametros** é muitas vezes crucial para que o modelo funcione!

Avaliação

- Usar os dados de treino para essa busca pode também causar overfitting. Portanto, um terceiro conjunto de **validação** é usado para essas escolhas



Avaliação

Exemplo: Classificação

```
model.eval()
val_loss, correct, total = 0.0, 0, 0

with torch.no_grad():
    for x_batch, y_batch in val_loader:
        y_hat = model(x_batch)
        loss = criterion(y_hat, y_batch)

        val_loss += loss.item()
        preds = y_hat.argmax(dim=1)
        correct += (preds == y_batch).sum().item()
        total += y_batch.size(0)

val_loss /= len(val_loader)
val_acc = correct / total
```

References

- https://docs.pytorch.org/tutorials/index.html
- https://cs231n.github.io/python-numpy-tutorial/
- pytorch-intro.ipynb — Façam uma cópia de dentro do google colab ou baixem o arquivo.